Step 1: Import Libraries

```
# @title Step 1: Import Libraries
import sklearn
import numpy as np
import pandas as pd
import nltk
from nltk.corpus import wordnet, stopwords
import matplotlib.pyplot as plt

nltk.download(['wordnet', 'stopwords', 'punkt', 'punkt_tab'])

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

Step 2: Datasets

```
# @title Step 2: Datasets
import pandas as pd

documents = [
    # Original
    "Online learning has changed the way students access education. It allows learners to attend classes from any location u

    # Few word changes
    "Online education has transformed how students get their studies. It lets learners join classes from anywhere using elec

    "Online learning has modified the way students receive education. It enables learners to attend classes from any place t

    "The system of online learning has changed student education. Learners can join classes from different locations using d

    # Paraphrased versions
    "Digital education platforms give students the freedom to study from any place. Through video lectures and online discus

    "Students today often rely on virtual classrooms and recorded materials for learning. These tools provide flexibility an

    "With the rise of internet-based education, students can access lessons remotely. Online platforms support communication

    "Education delivered through the internet allows learners to study at their own pace. Interactive tools and recorded con

    "Remote education systems provide students with flexible learning opportunities. Virtual lectures and discussions suppor

    # Completely different texts
    "Wildlife conservation protects endangered animals and their habitats. Human activities like deforestation and pollution

    "Cybersecurity involves protecting computer systems from digital attacks. Strong passwords, encryption, and firewalls he

    "Tourism plays a vital role in many countries’ economies. It creates jobs and promotes cultural exchange. Sustainable to

    "Eating nutritious food improves overall health and immunity. Balanced meals with vitamins and minerals reduce disease r

    "Solar and wind energy are clean alternatives to fossil fuels. They reduce pollution and help fight climate change. Many

    "Space technology supports satellite communication and weather forecasting. Research in space science helps us understan
]

df = pd.DataFrame({
    "doc_id": [f"Doc_{i+1}" for i in range(len(documents))],
    "text": documents
})

df.to_csv("text_similarity_dataset.csv", index=False)

df.head(15)
```

	doc_id	text
0	Doc_1	Online learning has changed the way students a...
1	Doc_2	Online education has transformed how students ...
2	Doc_3	Online learning has modified the way students ...
3	Doc_4	The system of online learning has changed stud...
4	Doc_5	Digital education platforms give students the ...
5	Doc_6	Students today often rely on virtual classroom...
6	Doc_7	With the rise of internet-based education, stu...
7	Doc_8	Education delivered through the internet allow...
8	Doc_9	Remote education systems provide students with...
9	Doc_10	Wildlife conservation protects endangered anim...
10	Doc_11	Cybersecurity involves protecting computer sys...
11	Doc_12	Tourism plays a vital role in many countries' ...
12	Doc_13	Eating nutritious food improves overall health...
13	Doc_14	Solar and wind energy are clean alternatives t...
14	Doc_15	Space technology supports satellite communicat...

Step 3: Preprocess

```
# @title Step 3: Preprocess
import string
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize

def preprocess_text(text):
    # 1. Lowercasing
    # Convert all characters in the text to lowercase. This ensures that words like 'The' and 'the'
    # are treated as the same word, which is crucial for consistency in text analysis.
    text = text.lower()
    print(f"Lowercase: {text[:100]}...") # Print a snippet for demonstration

    # 2. Punctuation Removal
    # Remove all punctuation marks from the text. Punctuation often doesn't carry semantic meaning
    # and can interfere with tokenization and other text analysis techniques.
    text = ''.join([char for char in text if char not in string.punctuation])
    print(f"Punctuation removed: {text[:100]}...")

    # 3. Tokenization
    # Break down the text into individual words or 'tokens'. This is a fundamental step that
    # converts a continuous string of text into a list of discrete units, making it suitable
    # for further linguistic processing.
    tokens = word_tokenize(text)
    print(f"Tokenization: {tokens[:15]}...")

    # 4. Stopword Removal
    # Eliminate common words (stopwords) like 'a', 'an', 'the', 'is', 'are', etc. These words
    # appear frequently but usually don't contribute significant meaning to the text.
    # Removing them reduces dimensionality and noise for many NLP tasks.
    stop_words = set(stopwords.words('english'))
    tokens = [word for word in tokens if word not in stop_words]
    print(f"Stopword removal: {tokens[:15]}...")

    # 5. Lemmatization (Optional)
    # Reduce words to their base or dictionary form (lemma). Unlike stemming, which chops off
    # prefixes/suffixes (e.g., 'running' -> 'run'), lemmatization uses vocabulary and morphological
    # analysis to return valid words (e.g., 'better' -> 'good'). This helps in normalizing words
    # and reducing inflections.
    lemmatizer = WordNetLemmatizer()
    lemmas = [lemmatizer.lemmatize(word) for word in tokens]
    print(f"Lemmatization: {lemmas[:15]}...")

    return lemmas

# Apply preprocessing to a sample document from the DataFrame
print("\n--- Applying Preprocessing to a Sample Document (Doc_1) ---")
sample_text = df['text'].iloc[0]
print(f"Original: {sample_text[:100]}...")
processed_tokens = preprocess_text(sample_text)

# You can also create a new column in your DataFrame for processed text
# This might take a moment depending on the size of your dataset
# df['processed_text'] = df['text'].apply(lambda x: ' '.join(preprocess_text(x)))
```

```
# print("\nDataFrame with processed text:")
# print(df[['doc_id', 'processed_text']].head())
```

```
--- Applying Preprocessing to a Sample Document (Doc_1) ---
Original: Online learning has changed the way students access education. It allows learners to attend classes ...
Lowercase: online learning has changed the way students access education. it allows learners to attend classes ...
Punctuation removed: online learning has changed the way students access education it allows learners to attend classes f...
Tokenization: ['online', 'learning', 'has', 'changed', 'the', 'way', 'students', 'access', 'education', 'it', 'allows', 'lea
Stopword removal: ['online', 'learning', 'changed', 'way', 'students', 'access', 'education', 'allows', 'learners', 'attend'
Lemmatization: ['online', 'learning', 'changed', 'way', 'student', 'access', 'education', 'allows', 'learner', 'attend', 'cl
```

Step 4: Feature Representation

```
# @title Step 4: Feature Representation
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from nltk.tokenize import word_tokenize
import pandas as pd

# Assuming df is already available and contains the 'text' column
# If df['processed_text'] was not created earlier, we will create it now.
# This ensures preprocessing is applied to all documents.

# First, ensure all documents are preprocessed into a list of tokens
def preprocess_text_for_vectorization(text):
    text = text.lower()
    text = ''.join([char for char in text if char not in string.punctuation])
    tokens = word_tokenize(text)
    stop_words = set(stopwords.words('english'))
    tokens = [word for word in tokens if word not in stop_words]
    lemmatizer = WordNetLemmatizer()
    lemmas = [lemmatizer.lemmatize(word) for word in tokens]
    return ' '.join(lemmas) # Join tokens back into a string for vectorizers

df['processed_text'] = df['text'].apply(preprocess_text_for_vectorization)
print("Processed text for vectorization:")
print(df[['doc_id', 'processed_text']].head())


# --- Bag-of-Words (BoW) for Jaccard Similarity ---

# 1. Create BoW Vectors
# We use binary=True to represent presence/absence of words, which is suitable for Jaccard similarity
count_vectorizer = CountVectorizer(binary=True)
bow_matrix = count_vectorizer.fit_transform(df['processed_text'])
bow_df = pd.DataFrame(bow_matrix.toarray(), columns=count_vectorizer.get_feature_names_out())

print("\nBag-of-Words (BoW) Matrix Sample (first 5 documents, first 10 features):")
print(bow_df.iloc[:5, :10])

# 2. Calculate Jaccard Similarity
# Jaccard similarity for two sets A and B is |A intersect B| / |A union B|
# For binary vectors, this can be calculated as: (A dot B) / (||A||^2 + ||B||^2 - (A dot B))
# A more direct way is to calculate it from sets of unique words.

def jaccard_similarity(set1, set2):
    intersection = len(set1.intersection(set2))
    union = len(set1.union(set2))
    return intersection / union if union != 0 else 0

# Convert processed text to sets of words for Jaccard calculation
df['processed_word_sets'] = df['processed_text'].apply(lambda x: set(x.split()))

jaccard_sim_matrix = pd.DataFrame(index=df['doc_id'], columns=df['doc_id'])

for i in range(len(df)):
    for j in range(len(df)):
        sim = jaccard_similarity(df['processed_word_sets'].iloc[i], df['processed_word_sets'].iloc[j])
        jaccard_sim_matrix.iloc[i, j] = sim

print("\nJaccard Similarity Matrix (first 5x5):")
print(jaccard_sim_matrix.iloc[:5, :5])


# --- TF-IDF for Cosine Similarity ---

# 1. Create TF-IDF Vectors
# TfidfVectorizer handles term frequency and inverse document frequency weighting
tfidf_vectorizer = TfidfVectorizer()
```

```
tfidf_matrix = tfidf_vectorizer.fit_transform(df['processed_text'])
tfidf_df = pd.DataFrame(tfidf_matrix.toarray(), columns=tfidf_vectorizer.get_feature_names_out())

print("\nTF-IDF Matrix Sample (first 5 documents, first 10 features):")
print(tfidf_df.iloc[:5, :10])

# 2. Calculate Cosine Similarity
# sklearn's cosine_similarity function takes two matrices and returns their pairwise cosine similarity
cosine_sim_matrix = cosine_similarity(tfidf_matrix)
cosine_sim_df = pd.DataFrame(cosine_sim_matrix, index=df['doc_id'], columns=df['doc_id'])

print("\nCosine Similarity Matrix (first 5x5):")
print(cosine_sim_df.iloc[:5, :5])
```

```
Processed text for vectorization:
  doc_id                                     processed_text
0  Doc_1  online learning changed way student access edu...
1  Doc_2  online education transformed student get study...
2  Doc_3  online learning modified way student receive e...
3  Doc_4  system online learning changed student educati...
4  Doc_5  digital education platform give student freedo...

Bag-of-Words (BoW) Matrix Sample (first 5 documents, first 10 features):
   academic  access  accessible  achieve  activity  advancement  affect  \
0         0       1           0        0         0            0       0
1         0       0           0        0         0            0       0
2         0       0           0        0         0            0       0
3         0       0           0        0         0            0       1
4         0       0           0        1         0            0       0

   allows  also  alternative
0       1     0            0
1       0     0            0
2       0     0            0
3       0     0            0
4       0     0            0

Jaccard Similarity Matrix (first 5x5):
doc_id     Doc_1     Doc_2     Doc_3     Doc_4     Doc_5
doc_id
Doc_1        1.0  0.358491      0.46  0.326923  0.157895
Doc_2   0.358491       1.0  0.258621  0.277778  0.137931
Doc_3       0.46  0.258621       1.0  0.272727  0.175439
Doc_4   0.326923  0.277778  0.272727       1.0     0.125
Doc_5   0.157895  0.137931  0.175439     0.125       1.0

TF-IDF Matrix Sample (first 5 documents, first 10 features):
   academic    access  accessible   achieve  activity  advancement    affect  \
0       0.0  0.178101         0.0  0.000000       0.0          0.0  0.000000
1       0.0  0.000000         0.0  0.000000       0.0          0.0  0.000000
2       0.0  0.000000         0.0  0.000000       0.0          0.0  0.000000
3       0.0  0.000000         0.0  0.000000       0.0          0.0  0.206446
4       0.0  0.000000         0.0  0.215126       0.0          0.0  0.000000

     allows  also  alternative
0  0.178101   0.0          0.0
1  0.000000   0.0          0.0
2  0.000000   0.0          0.0
3  0.000000   0.0          0.0
4  0.000000   0.0          0.0

Cosine Similarity Matrix (first 5x5):
doc_id     Doc_1     Doc_2     Doc_3     Doc_4     Doc_5
doc_id
Doc_1   1.000000  0.443334  0.570749  0.423545  0.223063
Doc_2   0.443334  1.000000  0.348094  0.340309  0.174209
Doc_3   0.570749  0.348094  1.000000  0.345497  0.227206
Doc_4   0.423545  0.340309  0.345497  1.000000  0.130089
Doc_5   0.223063  0.174209  0.227206  0.130089  1.000000
```

Step 5: Cosine Similarity

```
# @title Step 5: Cosine Similarity
print("\n--- Top 5 Most Similar Document Pairs (Cosine Similarity) ---")

# Create a list to store all unique similarity scores (excluding self-similarity)
similarity_scores = []

# Iterate through the upper triangle of the similarity matrix to avoid duplicates and self-similarity
for i in range(len(cosine_sim_df.index)):
    for j in range(i + 1, len(cosine_sim_df.columns)):
        doc1 = cosine_sim_df.index[i]
        doc2 = cosine_sim_df.columns[j]
        score = cosine_sim_df.iloc[i, j]
        similarity_scores.append((doc1, doc2, score))
```

```
# Sort the scores in descending order
similarity_scores.sort(key=lambda x: x[2], reverse=True)

# Highlight the top 5 most similar document pairs
for k, (doc1, doc2, score) in enumerate(similarity_scores[:5]):
    print(f"{k+1}. Pair: {doc1} and {doc2}, Similarity: {score:.4f}")

print("\n--- Interpretation of Cosine Similarity Scores ---")
print("Cosine similarity measures the cosine of the angle between two TF-IDF vectors. The closer the score is to 1.0, the m
print("\nBased on the analysis:")
print("- **High Scores (close to 1.0):** Indicate that the documents share a very similar thematic focus and vocabulary. Fo
print("- **Medium Scores (around 0.2 - 0.7):** Suggest some shared topic or vocabulary, but with significant differences in
print("- **Low Scores (close to 0):** Imply that the documents are largely dissimilar in terms of their content. Documents
print("\nThe observed top similar pairs generally reflect documents derived from the same original topic or its close varia
```

```
--- Top 5 Most Similar Document Pairs (Cosine Similarity) ---
1. Pair: Doc_1 and Doc_3, Similarity: 0.5707
2. Pair: Doc_1 and Doc_2, Similarity: 0.4433
3. Pair: Doc_1 and Doc_4, Similarity: 0.4235
4. Pair: Doc_2 and Doc_3, Similarity: 0.3481
5. Pair: Doc_3 and Doc_4, Similarity: 0.3455

--- Interpretation of Cosine Similarity Scores ---
Cosine similarity measures the cosine of the angle between two TF-IDF vectors. The closer the score is to 1.0, the more simi

Based on the analysis:
- **High Scores (close to 1.0):** Indicate that the documents share a very similar thematic focus and vocabulary. For exampl
- **Medium Scores (around 0.2 - 0.7):** Suggest some shared topic or vocabulary, but with significant differences in phrasir
- **Low Scores (close to 0):** Imply that the documents are largely dissimilar in terms of their content. Documents on compl

The observed top similar pairs generally reflect documents derived from the same original topic or its close variations, dem
```

Step 6: Jaccard Similarity

```
# @title Step 6: Jaccard Similarity
print("\n--- Top 5 Most Similar Document Pairs (Jaccard Similarity) ---")

# Create a list to store all unique similarity scores (excluding self-similarity)
jaccard_similarity_scores = []

# Iterate through the upper triangle of the similarity matrix to avoid duplicates and self-similarity
for i in range(len(jaccard_sim_matrix.index)):
    for j in range(i + 1, len(jaccard_sim_matrix.columns)):
        doc1 = jaccard_sim_matrix.index[i]
        doc2 = jaccard_sim_matrix.columns[j]
        score = jaccard_sim_matrix.loc[doc1, doc2] # Using .loc for DataFrame access
        jaccard_similarity_scores.append((doc1, doc2, score))

# Sort the scores in descending order
jaccard_similarity_scores.sort(key=lambda x: x[2], reverse=True)

# Highlight the top 5 most similar document pairs
for k, (doc1, doc2, score) in enumerate(jaccard_similarity_scores[:5]):
    print(f"{k+1}. Pair: {doc1} and {doc2}, Similarity: {score:.4f}")

print("\n--- Observation: Overlapping Word Sets and Jaccard Scores ---")
print("Jaccard Similarity is calculated as the size of the intersection of two sets of words divided by the size of their u
print("\nBased on the analysis:")
print("- **High Jaccard Scores (close to 1.0):** Indicate a very large overlap in the *unique words* used between the docum
print("- **Medium Jaccard Scores:** Suggest a noticeable but not overwhelming commonality in unique vocabulary. Paraphrased
print("- **Low Jaccard Scores (close to 0):** Point to very few, if any, unique words being shared between documents. Docum
print("\nThe observed top similar pairs using Jaccard similarity should generally align with the documents that share a hig
```

```
--- Top 5 Most Similar Document Pairs (Jaccard Similarity) ---
1. Pair: Doc_1 and Doc_3, Similarity: 0.4600
2. Pair: Doc_1 and Doc_2, Similarity: 0.3585
3. Pair: Doc_1 and Doc_4, Similarity: 0.3269
4. Pair: Doc_4 and Doc_6, Similarity: 0.3000
5. Pair: Doc_2 and Doc_4, Similarity: 0.2778

--- Observation: Overlapping Word Sets and Jaccard Scores ---
Jaccard Similarity is calculated as the size of the intersection of two sets of words divided by the size of their union. Es

Based on the analysis:
- **High Jaccard Scores (close to 1.0):** Indicate a very large overlap in the *unique words* used between the document pair
- **Medium Jaccard Scores:** Suggest a noticeable but not overwhelming commonality in unique vocabulary. Paraphrased documer
- **Low Jaccard Scores (close to 0):** Point to very few, if any, unique words being shared between documents. Documents on

The observed top similar pairs using Jaccard similarity should generally align with the documents that share a high number o
```

Step 7: WordNet Semantic Similarity

```
# @title Step 7: WordNet Semantic Similarity


# Re-use preprocessing logic, adapted to return a list of lemmas
def get_lemmas(text):
    text = text.lower()
    text = ''.join([char for char in text if char not in string.punctuation])
    tokens = word_tokenize(text)
    stop_words = set(stopwords.words('english'))
    tokens = [word for word in tokens if word not in stop_words]
    lemmatizer = WordNetLemmatizer()
    lemmas = [lemmatizer.lemmatize(word) for word in tokens]
    return lemmas

# Function to calculate WordNet similarity between two words
def word_similarity(word1, word2):
    synsets1 = wordnet.synsets(word1)
    synsets2 = wordnet.synsets(word2)

    max_similarity = 0.0
    if not synsets1 or not synsets2: # If no synsets found for either word
        return 0.0

    for syn1 in synsets1:
        for syn2 in synsets2:
            # Wu-Palmer Similarity is often a good choice for semantic relatedness
            similarity = syn1.wup_similarity(syn2)
            if similarity is not None and similarity > max_similarity:
                max_similarity = similarity
    return max_similarity

# Function to calculate WordNet similarity between two sentences
def sentence_similarity(sentence1, sentence2):
    lemmas1 = get_lemmas(sentence1)
    lemmas2 = get_lemmas(sentence2)

    if not lemmas1 or not lemmas2:
        return 0.0

    # Calculate the average best similarity score for words from sentence1 in sentence2
    # and vice versa, then average those two averages.
    # This handles sentences of different lengths and ensures all significant words are considered.

    # Part 1: How well does sentence1's words match sentence2's words?
    s1_to_s2_scores = []
    for l1 in lemmas1:
        max_s1_s2 = 0.0
        for l2 in lemmas2:
            sim = word_similarity(l1, l2)
            if sim > max_s1_s2:
                max_s1_s2 = sim
        s1_to_s2_scores.append(max_s1_s2)

    # Part 2: How well does sentence2's words match sentence1's words?
    s2_to_s1_scores = []
    for l2 in lemmas2:
        max_s2_s1 = 0.0
        for l1 in lemmas1:
            sim = word_similarity(l2, l1)
            if sim > max_s2_s1:
                max_s2_s1 = sim
        s2_to_s1_scores.append(max_s2_s1)

    # Combine scores, handle cases where lists might be empty if some lemmas had no matches
    avg_s1_to_s2 = np.mean(s1_to_s2_scores) if s1_to_s2_scores else 0.0
    avg_s2_to_s1 = np.mean(s2_to_s1_scores) if s2_to_s1_scores else 0.0

    # Return the average of the two directional averages
    return (avg_s1_to_s2 + avg_s2_to_s1) / 2

print("--- STEP 5: WordNet Semantic Similarity ---")

# Select at least 10 sentence pairs for demonstration
# doc_id is from the 'documents' list, starting from 1
selected_pairs = [
    ('Doc_1', 'Doc_2', "Original vs. Few word changes"),
    ('Doc_1', 'Doc_3', "Original vs. Few word changes"),
    ('Doc_1', 'Doc_4', "Original vs. Few word changes"),
    ('Doc_1', 'Doc_5', "Original vs. Paraphrased"),
```

```
    ('Doc_1', 'Doc_6', "Original vs. Paraphrased"),
    ('Doc_1', 'Doc_7', "Original vs. Paraphrased"),
    ('Doc_1', 'Doc_8', "Original vs. Paraphrased"),
    ('Doc_1', 'Doc_10', "Original vs. Completely different (Wildlife)"),
    ('Doc_1', 'Doc_11', "Original vs. Completely different (Cybersecurity)"),
    ('Doc_5', 'Doc_9', "Paraphrased vs. Paraphrased"),
    ('Doc_2', 'Doc_6', "Few word changes vs. Paraphrased"),
    ('Doc_10', 'Doc_11', "Completely different vs. Completely different") # Two unrelated docs
]

print("\nCalculating semantic similarities...")
results = []
for doc_id1, doc_id2, description in selected_pairs:
    text1 = df[df['doc_id'] == doc_id1]['text'].iloc[0]
    text2 = df[df['doc_id'] == doc_id2]['text'].iloc[0]
    similarity = sentence_similarity(text1, text2)
    results.append({'Pair': f"{doc_id1} vs. {doc_id2}", 'Description': description, 'Similarity': similarity})

results_df = pd.DataFrame(results)
results_df_sorted = results_df.sort_values(by='Similarity', ascending=False).reset_index(drop=True)

print("\n--- WordNet Semantic Similarity Scores ---")
print(results_df_sorted.to_string(index=False))

print("\n--- Interpretation of WordNet Semantic Similarity Scores ---")
print("WordNet semantic similarity attempts to quantify how close in meaning two words or sentences are, based on their pos
print("\nBased on the analysis:")
print("- **High Scores (e.g., >0.7):** Indicate a strong semantic overlap. These pairs often represent synonyms, direct par
print("- **Medium Scores (e.g., 0.3 - 0.7):** Suggest some shared conceptual ground, but with noticeable differences in spe
print("- **Low Scores (e.g., <0.3):** Imply that the sentences are semantically distinct. Documents on entirely different t
print("\nWordNet similarity is particularly good at detecting meaning similarity even when direct word overlap is low, whic
```

```
--- STEP 5: WordNet Semantic Similarity ---

Calculating semantic similarities...

--- WordNet Semantic Similarity Scores ---
             Pair                                      Description  Similarity
  Doc_1 vs. Doc_3                    Original vs. Few word changes    0.938494
  Doc_1 vs. Doc_2                    Original vs. Few word changes    0.926009
  Doc_1 vs. Doc_4                    Original vs. Few word changes    0.879923
  Doc_1 vs. Doc_6                         Original vs. Paraphrased    0.832969
  Doc_2 vs. Doc_6                  Few word changes vs. Paraphrased    0.830766
  Doc_1 vs. Doc_5                         Original vs. Paraphrased    0.828750
  Doc_5 vs. Doc_9                       Paraphrased vs. Paraphrased    0.827281
  Doc_1 vs. Doc_7                         Original vs. Paraphrased    0.823815
  Doc_1 vs. Doc_8                         Original vs. Paraphrased    0.815041
 Doc_1 vs. Doc_10      Original vs. Completely different (Wildlife)    0.709238
 Doc_1 vs. Doc_11 Original vs. Completely different (Cybersecurity)    0.688561
Doc_10 vs. Doc_11     Completely different vs. Completely different    0.665577

--- Interpretation of WordNet Semantic Similarity Scores ---
WordNet semantic similarity attempts to quantify how close in meaning two words or sentences are, based on their position ir

Based on the analysis:
- **High Scores (e.g., >0.7):** Indicate a strong semantic overlap. These pairs often represent synonyms, direct paraphrases
- **Medium Scores (e.g., 0.3 - 0.7):** Suggest some shared conceptual ground, but with noticeable differences in specific de
- **Low Scores (e.g., <0.3):** Imply that the sentences are semantically distinct. Documents on entirely different topics sh

WordNet similarity is particularly good at detecting meaning similarity even when direct word overlap is low, which is its k
```

Step 8: Comparison Section

```
# @title Step 8: Comparison Section

# Load dataset
df = pd.read_csv("text_similarity_dataset.csv")

# ---------------- PREPROCESSING ----------------
stop_words = set(stopwords.words('english'))

def preprocess(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', '', text)
    tokens = word_tokenize(text)
    tokens = [w for w in tokens if w not in stop_words]
    return tokens

df["tokens"] = df["text"].apply(preprocess)

# ---------------- COSINE SIMILARITY (TF-IDF) ----------------
vectorizer = TfidfVectorizer(stop_words='english')
tfidf_matrix = vectorizer.fit_transform(df["text"])
```

```
tfidf_matrix = vectorizer.fit_transform(df["text"])
cosine_sim_matrix = cosine_similarity(tfidf_matrix)

# ---------------- JACCARD SIMILARITY (BoW) ----------------
def jaccard_similarity(set1, set2):
    return len(set1 & set2) / len(set1 | set2)

token_sets = df["tokens"].apply(set)

jaccard_matrix = np.zeros((len(token_sets), len(token_sets)))
for i in range(len(token_sets)):
    for j in range(len(token_sets)):
        jaccard_matrix[i][j] = jaccard_similarity(token_sets[i], token_sets[j])

# ---------------- WORDNET ENHANCED JACCARD ----------------
def get_synonyms(word):
    synonyms = set()
    for syn in wordnet.synsets(word):
        for lemma in syn.lemmas():
            synonyms.add(lemma.name().lower().replace("_", " "))
    return synonyms

def expand_with_synonyms(word_set):
    expanded = set(word_set)
    for word in word_set:
        expanded |= get_synonyms(word)
    return expanded

expanded_sets = token_sets.apply(expand_with_synonyms)

wordnet_matrix = np.zeros((len(expanded_sets), len(expanded_sets)))
for i in range(len(expanded_sets)):
    for j in range(len(expanded_sets)):
        wordnet_matrix[i][j] = jaccard_similarity(expanded_sets[i], expanded_sets[j])

# ---------------- DISPLAY RESULTS ----------------
doc_ids = df["doc_id"].tolist()

cosine_df = pd.DataFrame(cosine_sim_matrix, index=doc_ids, columns=doc_ids)
jaccard_df = pd.DataFrame(jaccard_matrix, index=doc_ids, columns=doc_ids)
wordnet_df = pd.DataFrame(wordnet_matrix, index=doc_ids, columns=doc_ids)

print("🔹 Cosine Similarity (TF-IDF)")
display(cosine_df.round(2))

print("🔹 Jaccard Similarity (BoW)")
display(jaccard_df.round(2))

print("🔹 WordNet Enhanced Jaccard Similarity")
display(wordnet_df.round(2))
```

Step 9: Lab Report Section

```
# @title Step 9: Lab Report Section
import pandas as pd
import numpy as np
import re
import nltk
from nltk.corpus import stopwords, wordnet
from nltk.tokenize import word_tokenize
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Download required resources
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')

# Load dataset
df = pd.read_csv("text_similarity_dataset.csv")

# ---------------- PREPROCESSING ----------------
stop_words = set(stopwords.words('english'))

def preprocess(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', '', text)
    tokens = word_tokenize(text)
    tokens = [w for w in tokens if w not in stop_words]
```

```
    return tokens

df["tokens"] = df["text"].apply(preprocess)

# ---------------- COSINE SIMILARITY (TF-IDF) ----------------
vectorizer = TfidfVectorizer(stop_words='english')
tfidf_matrix = vectorizer.fit_transform(df["text"])
cosine_sim = cosine_similarity(tfidf_matrix)

# ---------------- JACCARD SIMILARITY ----------------
def jaccard_similarity(a, b):
    return len(a & b) / len(a | b)

token_sets = df["tokens"].apply(set)
jaccard_sim = np.zeros((len(df), len(df)))

for i in range(len(df)):
    for j in range(len(df)):
        jaccard_sim[i][j] = jaccard_similarity(token_sets[i], token_sets[j])

# ---------------- WORDNET ENHANCED JACCARD ----------------
def get_synonyms(word):
    synonyms = set()
    for syn in wordnet.synsets(word):
        for lemma in syn.lemmas():
            synonyms.add(lemma.name().lower().replace("_", " "))
    return synonyms

def expand_set(word_set):
    expanded = set(word_set)
    for word in word_set:
        expanded |= get_synonyms(word)
    return expanded

expanded_sets = token_sets.apply(expand_set)

wordnet_sim = np.zeros((len(df), len(df)))
for i in range(len(df)):
    for j in range(len(df)):
        wordnet_sim[i][j] = jaccard_similarity(expanded_sets[i], expanded_sets[j])

# ---------------- REPORT OUTPUT ----------------
print("\n===== LAB REPORT RESULTS SUMMARY =====\n")

print("1. Objective:")
print("This experiment evaluates multiple text similarity techniques to detect copied, modified, and paraphrased documents.

print("2. Dataset Description:")
print(f"The dataset contains {len(df)} documents including original text, modified copies, paraphrased versions, and unrela

print("3. Preprocessing Steps:")
print("Text was converted to lowercase, punctuation removed, stopwords filtered, and tokenization applied.\n")

print("4. Cosine Similarity (TF-IDF) Results:")
print("Average cosine similarity (excluding self-comparison):",
      round((cosine_sim.sum() - len(df)) / (len(df)*(len(df)-1)), 3))
print("Higher values were observed among original and modified documents.\n")

print("5. Jaccard Similarity Results:")
print("Average Jaccard similarity:",
      round((jaccard_sim.sum() - len(df)) / (len(df)*(len(df)-1)), 3))
print("This method showed strong similarity mainly for directly copied texts.\n")

print("6. WordNet Enhanced Results:")
print("Average WordNet-enhanced similarity:",
      round((wordnet_sim.sum() - len(df)) / (len(df)*(len(df)-1)), 3))
print("Similarity improved for paraphrased documents due to synonym matching.\n")

print("7. Comparison:")
print("Cosine similarity performed best for detecting paraphrased content.")
print("Jaccard similarity worked best for exact or near copies but failed with synonyms.")
print("WordNet helped bridge this gap by recognizing semantic similarity.\n")

print("8. Conclusion:")
print("Combining TF-IDF cosine similarity, Jaccard similarity, and WordNet enhancement provides a more reliable plagiarism
```

Doc_12	0.04	0.03	0.03	0.02	0.02	0.00	0.02	0.01	0.00	0.00	0.05	1.00	0.02	0.08	0.04
Doc_13	0.07	0.02	0.02	0.02	0.02	0.00	0.02	0.00	0.13	0.02	0.11	0.01	0.01	0.02	1.00	0.09	0.14
Doc_14	0.11	0.03	0.07	0.03	0.02	0.02	0.00	0.00	0.01	0.02	0.09	0.08	0.09	1.00	0.05
Doc_15	0.03	0.03	0.10	0.01	0.01	0.02	0.11	0.03	0.10	0.00	0.04	0.04	0.14	0.05	1.00

```
===== LAB REPORT RESULTS SUMMARY =====

1. Objective:
This experiment evaluates multiple text similarity techniques to detect copied, modified, and paraphrased documents.
```

```
2. Dataset Description:
The dataset contains 15 documents including original text, modified copies, paraphrased versions, and unrelated texts.

3. Preprocessing Steps:
Text was converted to lowercase, punctuation removed, stopwords filtered, and tokenization applied.

4. Cosine Similarity (TF-IDF) Results:
Average cosine similarity (excluding self-comparison): 0.078
Higher values were observed among original and modified documents.

5. Jaccard Similarity Results:
Average Jaccard similarity: 0.061
This method showed strong similarity mainly for directly copied texts.

6. WordNet Enhanced Results:
Average WordNet-enhanced similarity: 0.097
Similarity improved for paraphrased documents due to synonym matching.

7. Comparison:
Cosine similarity performed best for detecting paraphrased content.
Jaccard similarity worked best for exact or near copies but failed with synonyms.
WordNet helped bridge this gap by recognizing semantic similarity.

8. Conclusion:
Combining TF-IDF cosine similarity, Jaccard similarity, and WordNet enhancement provides a more reliable plagiarism detectic

[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
[nltk_data]   Package omw-1.4 is already up-to-date!
```